

La programmazione asincrona (approccio ad eventi) è uno stile duale (ortogonale) rispetto alla programmazione basata su thread perchè entrambi gli approcci risolvono lo stesso problema (la gestione di attività concorrenti e non bloccanti) ma lo fanno adottando punti di vista e meccanismi opposti:

- I Thread rappresentano l'idea di modellare un programma concorrente con più processi che possono non terminare e possono interagire tra loro attraverso memoria condivisa.

- Invece, in Programmazione Asincrona, si ha un solo processo in foreground (event loop) che tipicamente non termina mai, perché è in attesa di eventi, ed esegue gli handler che sono visti come unità di lavoro (Task) che devono terminare affinché l'event loop ritorni a reagire (in background ci sono dei Thread, in un'ottica Master-Worker, a cui vengono assegnati quei compiti che sarebbero bloccanti che vengono delegati ai BG Worker)

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[module-2.1]

ASYNCHRONOUS
PROGRAMMING

ASYNCHRONOUS PROGRAMMING

Thread è una virtualizzazione di una CPU/flusso di controllo (descrivere un programma come se la CPU fosse tutta sua, ha un proprio flusso di controllo che viene mappata sulle CPU)

- A style of programming getting more and more important today
 - generally speaking, it's about ~~about~~ the execution and management of *asynchronous computations/request/processes*
 - abstracting from threads...
 - recently supported by all major programming frameworks and mainstream platforms
 - Java, .NET, iOS, Android..
 - including a variety of mechanisms and architectures
 - including event-driven programming, but not limited to
- Still an evolving issue
 - e.g. Microsoft case: first proposing Asynchronous Programming Model (APM), deprecated then for the Event-based Asynchronous Pattern (EAP) with .NET 2.0, deprecated for the Task-based Asynchronous Pattern (TAP) with .NET 4.5...
 - <https://docs.microsoft.com/en-us/dotnet/standard/asynchronous-programming-patterns/>

Gli handler sono delle computazioni tipicamente brevi, in modo tale da ritornare disponibili a reagire agli eventi esterni (sicuramente, non possono andare in loop infiniti: non ho più un componente reattivo)

Vogliamo un sistema che dati degli eventi/input che arrivano in modo asincrono, vogliamo che reagisca prontamente ed esegua delle azioni/computazioni in reazione a questi eventi. Sistema che dipende ma è reattivo dall'esterno.

CURRENT ASYNC PANORAMA

Si dice che è il "motore" perché descrive come il sistema lavora internamente. In un modello event-driven, il programma non aspetta pigramente che un'operazione finisca; rimane in un ciclo infinito (l'Event Loop) pronto a reagire a segnali esterni (I/O, click, timer).

Motore (Engine)

Event-driven Programming

- ~~event loop based control flow architectures~~
- reactor pattern

È il design pattern architetturale che definisce come si organizza l'event loop

Modello di Programmazione (non modificano il motore)

Async Functions & Continuation Passing Style

- event handlers as continuations
- promises library support

Async/await language mechanism

- mimicking synchronous being asynchronous

Coroutine-based concurrency

- lightweight threads

Sono i linguaggi e le strutture che noi programmatori usiamo per interagire con il motore ad eventi senza impazzire nella gestione della complessità.

Idea di base -> Sistemi reattivi: parto dall'idea che organizzo il programma come un insieme di Handler con cui voglio reagire ad eventi. Questi handler vengono eseguiti sempre e solo da un Thread (l'event loop). La scrittura di questi handler ha dei vincoli forti: un handler non si può mai bloccare/inlooparsi perché deve essere eseguito velocemente (l'unico punto in cui event loop può bloccarsi è quando torna a sospendersi in attesa di vedere un evento). Gli eventi vengono elaborati sequenzialmente, ma quando ci sono chiamate sincrone o di I/O, il compito viene delegato a un Thread che noi non vediamo (simile a Master-Worker). Si usa come unico modello di interazione con event loop, la coda degli eventi: così che event loop può reagire sia ad eventi generati da esterno sia ad eventi generati da background thread

Cosa succede se arriva un evento, mentre sto eseguendo un Handler? L'evento viene messo nella coda (quindi, ho l'esterno che mi produce gli eventi in modo asincrono e me li mette nella coda ed il sistema che sta elaborando gli eventi in modo asincrono sa che non perde gli eventi, se la coda è sufficientemente grande, così da elaborare ogni evento uno alla volta). Quindi, gli eventi vengono messi nella coda ed in modo asincrono processati. Ci sono dei problemi di concorrenza eseguendo due Handler contemporaneamente che generano corse critiche? Se tengo un solo event loop, con una sola coda, e vincolo il fatto che sia lui ad elaborare tutti gli eventi ed eseguire gli handler, sono sicuro che non possono esserci corse critiche. Se avessi più event loop in attesa sulla stessa coda, avrei problemi di CS. Quindi, event loop con la sua coda è sempre una sola.

L'Event Loop ti garantisce che gli handler non si scontrino mentre "scrivono" fisicamente il dato (no CS), ma non ti garantisce che i dati che hanno letto all'inizio siano ancora validi se nel mezzo c'è stata una sospensione (await).

Come applico questo modello (Async Programming) quando un Handler deve fare operazioni bloccanti (tipo Leggere un File), dove il dogma è "non mi devo mai bloccare"? Ad aiutare l'event loop (Master Worker), immagina che ci sia un pool di worker (background thread: loro si possono bloccare a differenza dell'event loop) che noi non vediamo che quando devo fare un'operazione bloccante, il handler si sospende e delega l'operazione bloccante a qualcuno del Pool di Worker e l'event loop, che è di nuovo libero, perché l'handler si è sospeso, si mette in attesa sulla coda degli eventi. Quando il Worker finisce di leggere il file. Non può chiamare direttamente l'Event Loop, perché sta eseguendo un altro handler, allora prende il risultato e lo mette nella Coda degli Eventi. L'Event Loop, appena finisce l'esecuzione del handler, guarda la Coda, vede l'evento del file pronto ed il handler associato (che prende la seconda parte dell'Handler originale (quella che aspettava i dati)) e la esegue.

EVENT-DRIVEN PROGRAMMING

Esecuzione degli Handler è non-deterministico (dipende dagli eventi provenienti dall'ambiente)

- A program is made of routines (**event handlers**) that are called (executed, triggered) when **events** occur
 - events can refer to any kind of (asynchronous) change of state or input from the environment which is relevant for the program
 - examples: user actions (mouse clicks, key presses), sensor outputs, messages, replies received...
 - ~~the control flow of the program depends on events~~
 - non-determinism
 - “programming without a call stack”
 - after each handler execution the stack is empty
- Aspects
 - ~~how handlers are associated to events~~ (come associo un handler ad un evento?)
 - ~~how events are received and managed~~ (quale struttura dati usare per gestire e ricevere gli eventi?)
 - ~~how handlers are executed~~ (come vengono mandati in esecuzione gli handler?)

EVENTS VS. THREADS: AN OLD DEBATE

- Across different topics
 - *Why threads are a bad idea (for most purposes)*. John Ousterhout - Invited Talk at the 1996 USENIX Technical Conference
 - *The Problem with Threads* (Edward Lee) - IEEE Computer, Vol. 39, No. 5, pp. 33–42, May 2006
 - *Why events are a bad idea (for high-concurrency servers)* (von Behren, Condit and Brewer) Proceedings of the 9th USENIX Conference on Hot Topics in Operating Systems, 2003
 - *Cooperative Task Management without Manual Stack Management - or, Event-driven Programming is Not the Opposite of Threaded Programming*. Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky, John R. Douceur. Microsoft Research. Proceedings of the 2002 Usenix Annual Technical Conference, Monterey, CA, June 2002.
- Each approach has pros and cons
 - the real challenge is about how to put them together...

Si chiama "di controllo" perché l'Event Loop è il regista dell'intera esecuzione. Gli handler sono solo "attori" che entrano in scena, recitano la loro parte (atomica) e se ne vanno, restituendo immediatamente il copione al regista.

EXECUTION MODEL: "EVENT LOOP"

L'architettura di flusso di controllo:

Event loop control architecture (abstract model)

Viene chiamata architettura di controllo perché l'Event Loop non è semplicemente una parte del codice, ma è l'entità che detiene il potere decisionale su cosa deve essere eseguito e quando.

- *single thread of control* waiting for events and executing the handler
- an event queue used to keep track of events generated by either the environment or the event handler themselves

```
loop {  
    Event ev = waitForEvent(eventQueue)  
    Handler handler = selectHandler(ev)  
    execute(handler)  
}
```

- Event handlers are executed **atomically**
 - if an event occurs while an handler is in execution, the corresponding event handler is executed only when the handler in execution has completed (no concurrency)

Con un solo event loop, ho la garanzia che non ci saranno mai corse critiche: quando viene eseguito un Handler ed esso accede ad oggetti passivi nella zona di memoria, sono sicuro che quell'accesso viene eseguito in modo mutuamente esclusivo perché c'è solo l'event loop che esegue in quel punto (ogni esecuzione di un handler, nell'event loop, è atomica rispetto agli altri handler) (i background thread non hanno accesso alla stessa memoria condivisa dei Handler: l'oggetto passivo in memoria viene aggiornato solo dall'Event Loop quando processa il messaggio di ritorno. Quindi, il Worker non accede alla stessa memoria del Handler). Ho la garanzia che non possono esserci corse critiche nell'esecuzione dell'event loop (perché è un solo Thread)

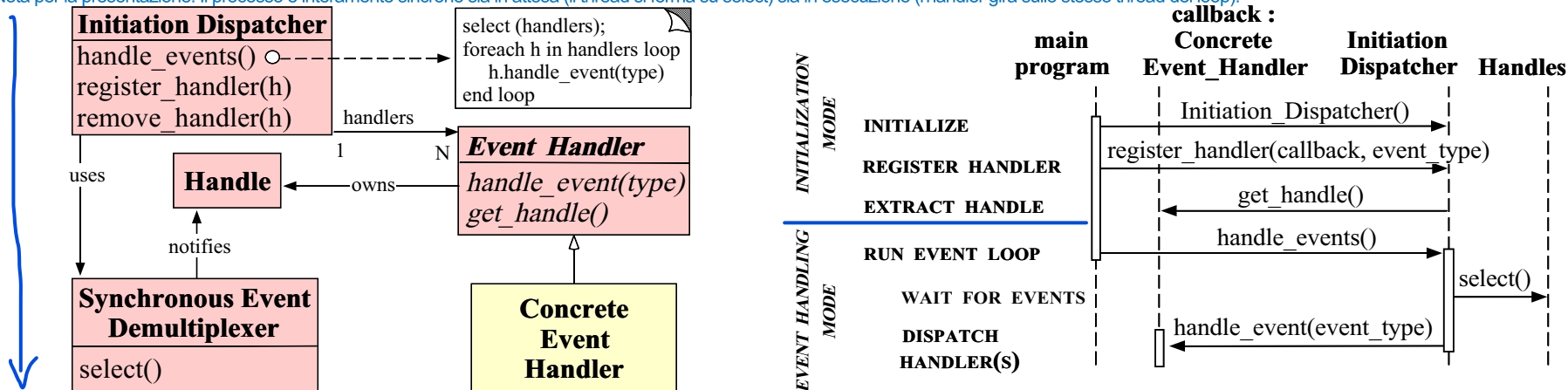
RELATED: REACTOR PATTERN

Il Reactor Pattern è il Design Pattern architetturale che l'event loop mette in pratica

Il Flusso di Controllo del Reactor Pattern

1. Initialization Mode (Configurazione): Inizializzazione: Il main program crea l'Initiation Dispatcher. Registrazione: L'applicazione registra il proprio Concrete Event Handler (callback) associandolo a uno specifico tipo di evento. Estrazione dell'Handle: Il Dispatcher ricava la risorsa interna (Handle) da monitorare a basso livello (es. un descrittore di socket).
2. Event Handling Mode (Ciclo di Reazione): Avvio dell'Event Loop: Il main program cede il controllo definitivo al Dispatcher invocando `handle_events()`. Attesa Sincrona (Il Blocco): Il Dispatcher interroga il Synchronous Event Demultiplexer tramite la chiamata nativa `select()`. Il thread si blocca fisicamente in attesa che il sistema operativo segnali una risorsa pronta. Dispacciamento ed Esecuzione: Non appena un evento si sblocca, il Dispatcher intercetta la notifica e "spara" la chiamata (dispatch) eseguendo in modo sequenziale la funzione `handle_event(type)` sul relativo Handler.

Nota per la presentazione: Il processo è interamente sincrono sia in attesa (il thread si ferma su `select`) sia in esecuzione (l'handler gira sullo stesso thread del loop).



La Registrazione: Quando l'applicazione chiama `register_handler(callback, event_type)`, sta passando l'oggetto `Concrete_Event_Handler` intero all'Initiation Dispatcher. L'Estrazione: Il Dispatcher chiama immediatamente `get_handle()` sul `Concrete_Event_Handler` per farsi restituire la risorsa associata (l'Handle). A questo punto, il Dispatcher salva internamente una mappa dove la chiave è l'Handle, e il valore è l'Handler stesso (l'oggetto concreto, non solo la funzione di callback). L'Attesa: Il Dispatcher passa gli Handle nudi e crudi al Demultiplexer (`select()`). Il Risveglio: La `select()` si sblocca e restituisce al Dispatcher solo l'Handle (o la lista di Handle) su cui è attivo un evento. Il Demultiplexer non sa nulla degli handler dell'applicazione, conosce solo i file descriptor del sistema operativo. Il Dispatch reale: Il Dispatcher prende l'Handle restituito dal Demultiplexer, lo usa come chiave per pescare il `Concrete_Event_Handler` dalla sua mappa interna, e invoca `handle_event(event_type)` direttamente sul `Concrete_Handler`.

- Schmidt, Douglas et al. Pattern-Oriented Software Architecture Volume 2: Patterns for Concurrent and Networked Objects. Volume 2. Wiley, 2000.
- Schmidt, Douglas C., An Object Behavioral Pattern for Demultiplexing and Dispatching Handles for Synchronous Events - <http://www.dre.vanderbilt.edu/~schmidt/PDF/reactor-siemens.pdf>

Invece di creare un thread per ogni descrittore (spreca memoria e tempo di CPU), il Reactor raccoglie tutti gli Handle registrati dall'applicazione e li passa in blocco al Demultiplexer. Il Demultiplexer riduce la molteplicità ad unità: prende un insieme di N canali potenzialmente pronti, si mette in attesa bloccante su tutti contemporaneamente e, non appena uno o più canali si attivano, si sveglia e restituisce al Dispatcher solo la lista ristretta degli Handle che hanno effettivamente ricevuto un evento.

MULTIPLE EVENT LOOPS

- Conceptually, *an event-loop can be used as the control architecture of a single active component*
 - event-driven behaviour
 - private event queue & asynchronous non blocking tasks
- A system can be composed by *multiple active components each one with its own event-loop*
 - not sharing memory perchè non voglio usare mai i meccanismi che uso per i Thread (come i semafori)
 - interacting by generating events (Ogni event-loop ha la propria memoria non condivisa e per comunicare generano gli eventi che vengono messi nelle code dei vari event-loop interessati)

NEVER-BLOCKING RULE & CONCURRENCY

- The execution model leads to an important rule in designing/programming event handlers:

event handlers should never blocks (should not contain blocking calls) and should always terminate (must not contain infinite loops)

- This is to guarantee *responsiveness* of the system
 - a blocking call or infinite loop would block the event loop serving the events and events in the queue would no be processed
 -  *quindi* a blocking call must be replaced by *an asynchronous request or computation*, which will eventually generate some event in the future (result or error)
 - asynchronous requests/tasks are served by other threads, independent from the event loop
 - these threads interact with the event loop by inserting events in the event queue

FROM EVENT HANDLERS TO ASYNC FUNCTIONS & CALLBACKS

La soluzione: Non eseguire il compito dentro l'handler, ma "delegarlo".
L'Esecuzione Asincrona
L'handler non fa il lavoro pesante.
Invece:
- Invia una richiesta ad un pool di thread separato (es. "Ehi, leggi questo file").
- Esce immediatamente.
- L'Event Loop è di nuovo libero di gestire altri eventi.



- Given the no-blocking rule, a main issue is:
 - *how to execute long-running handlers, without blocking and ^{compromettere} tampering reactivity?*
- => **by executing asynchronous tasks / functions**
 - the handler triggers the execution of an async task/function and goes on, without waiting for the result/error

- Key point/problem

how to process results or errors that are asynchronously generated by the async task / function execution?

- => **callback model** (viene generato un evento, dal BG worker, a cui viene associato un Handler che lo gestisca)

Uno dei grandi vantaggi dell'event-loop è non avere corse critiche a priori. Il vantaggio del modello di programmazione basato su CPS è che invece di frammentare il programma in tanti spaghetti separati, tutto diventa compatto usando le callback che permettono di avere modularità

CALLBACK MODEL & CONTINUATION PASSING STYLE

Modello generale: ogni funzione sincrona che voi chiamereste in un handler viene trasformata in una funzione/routine asincrona in cui viene aggiunto un parametro che sarebbe l'handler che specifica come deve essere gestito il risultato (cioè come continuare dopo che il risultato è arrivato). Questo stile di programmazione prende il nome di CPS

- A function (**callback**) is specified as an argument (last argument) of the async task / function call
- The callback is called by the event loop when (if) processing the event related to the result or error generated by the async task / function execution
 - it is a kind of “implicit” handler registered dynamically to the event corresponding to the result/error
- In this way the callback defines a “*continuation*” of the computation

=> **Continuation-Passing Style (CPS)** stile di programmazione introdotto per fare in modo di evitare di usare lo stack

la continuazione è chiamata callback solo in Programmazione Asincrona (programmazione asincrona si basa su CPS). In CPS, rimane con lo stesso nome

Esempio: immagina un programma sequenziale che dato un input alla funzione restituisce un output (dato x, incremento x di 1 e restituisco). Un programma basato su CPS non ha più funzioni che restituiscono qualcosa: questo significa che ogni funzione, del programma sequenziale, la devo trasformare dimenticando che posso tornare indietro (in CPS, posso andare solo in avanti). Quindi, anzichè chiamare la funzione 'successore' e farmi restituire il numero e poi stamparlo, passo la continuazione come parametro della funzione e, senza tornare indietro e ritornare il valore, stampo direttamente il risultato dopo che ho il risultato di x+1

Considerando un'esecuzione su event-loop, è deterministico (univocamente definito) l'ordine con cui vengono eseguite le istruzioni A,B,C o no?

(HANDLER)

[A] x=0

async task (..., [B] callback)

[C] x=x+1

Risposta: l'ordine è deterministico/univocamente definito (inoltre, non è possibile avere corse critiche) perché event-loop sta già eseguendo l'Handler ed anche se il Thread legato all'async task è velocissimo e finisce prima di [C] comunque l'Handler della callback non viene eseguito direttamente dal Background Worker (Gli handler vengono sempre eseguite da event-loop, stessa cosa quelli associati a callback). Quindi, event-loop finisce di eseguire un Handler alla volta e poi va a prendere il prossimo dalla Coda (in questo caso, l'Handler di [B]), così da non avere nemmeno corse critiche (in caso non ci sono eventi nella coda, l'event loop si sospende sulla coda aspettando evento). Quindi, l'ordine è deterministico (A C B)

CONTINUATION PASSING STYLE (CPS)

Non è nato per la concorrenza, ma utile usarlo

- In functional programming, continuation-passing style (CPS) is a style of programming in which control is passed explicitly in the form of a *continuation*
 - first introduced by Gerald Jay Sussman and Guy L. Steele, Jr. 1975 working on Scheme
- Popular intermediate format for compilers of function languages,
 - many control flow constructs can be elegantly expressed
 - easy tail call optimization
- “Stack-less” programming perchè non ho bisogno di tornare indietro (usare lo stack per tornare alla chiamata precedente), vado solo in avanti

CPS IN GENERAL

- A function written in continuation-passing style takes an extra argument, a function of one argument i.e. *the continuation*
- When the CPS function has computed its result value, it "returns" it by calling the continuation function with this value as the argument
 - so the stack is not used, actually
- when invoking a CPS function, ^{la funzione chiamante} ~~the calling function~~ is required to supply a procedure to be invoked with the subroutine's ^{result} ~~"return"~~ value

```
void sum(x, y, cont){  
    cont(x + y);  
}
```

```
sum(1, 3, (res) -> {  
    print(res)  
});
```

CALLBACKS AS CONTINUATIONS

Le callback sono sempre eseguite da event loop, non dai Thread (non si passa la callback ai Background Worker). Quindi, non ci sono corse critiche

- *callbacks* are like continuations that are called when the result of the asynchronous computation is ready
 - examples:
 - the response of an asynchronous request to a remote service has arrived
 - the result of a I/O task is available
 - ...
- Abstract example in JavaScript:

// **synch version** Modello Sincrono (ho due chiamate bloccanti)

record user
function loadUserPic(userId) {
 let user = findUserById(userId);
 return loadPic(user.picId);
}

Come trasformo il Modello Sincrono in modo che possa essere usato nell'architettura con event loop?

In questo esempio, scompaiono gli eventi, ma sotto il motore è ad eventi (gestito da event loop)
/* **async version** -
both **findUserById** & **loadUserPic**
are **async processes** */

function loadUserPic(userId, ret) {
 findUserById è asincrona (secondo arg. è una callback)
 findUserById(userId, (user) => {
 loadPic(user.picId, ret);
 }); Quando ho il record del user, vado a chiamare loadPic (anch'essa asincrona)
}

Quando sarà pronto il risultato (la picture), mostrala sulla UI
(callback: handler da mandare in esecuzione dopo che ho la picture)
loadUserPic('john', (pic) => {
 ui.show(pic);
}); Non sto chiamando la funzione 'ret', sto definendo l'handler che deve essere eseguito quando io ricevo l'evento "async task è stato concluso" (quindi findUserById e loadPic che hanno entrambi un callback)

USING CLOSURES AS CALLBACKS

Closure (Callback Asincrona) = Codice della Funzione + Ambiente (Environment)

Callbacks are often implemented as *closures*

- technique for implementing lexically scoped name binding in a language with first-class functions. una funzione è in grado di "vedere" e ricordare le variabili che si trovano attorno a lei nel codice in cui è stata scritta (lo scope lessicale).
- operationally, a closure is a record storing a function[a] together with an environment.
 - the environment is a mapping associating each free variable of the function with the value or reference to which the name was bound when the closure was created

Cos'è una Closure? Una closure è un "pacchetto" composto da due parti:
- Il Codice: La callback stessa.
- L'Ambiente (Environment): Una mappa (un "fermo immagine") di tutte le variabili che erano visibili nel momento in cui la funzione è stata creata.

Quando crei la callback, il linguaggio non "scrive" solo il codice, ma "cattura" le variabili esterne e le salva in questo record persistente.

La callback riesce a vedere e ricordare le sue variabili locali e i suoi parametri (Quelli definiti normalmente dentro la callback stessa) e le variabili esterne dello scope lessicale (Tutte le variabili, i parametri o le funzioni che si trovano all'esterno della callback, ma dentro la funzione che la circonda nel momento in cui la callback viene scritta (il codice circostante)).

CALLBACK EXECUTION MODEL

- A main question is:

who invokes the continuation/callback?

- Two main possibilities:

1. a thread of control *different from the thread that triggered the request* (e.g., the thread generating the event)

– problems: races

2. in event-driven architecture based on the event loop model, *the continuation is executed by the event loop, i.e. the logical thread of control that triggered the async task / function* (which is different from the thread that served/executed the async task / function)

- execution model of modern web apps (client/server)

- L'esecutore è l'Event Loop: La continuazione viene eseguita dall'Event Loop, ovvero il flusso di controllo logico (thread principale) che ha originariamente fatto partire la funzione o il task asincrono.

- La separazione dei thread: Il thread dell'Event Loop (che esegue la callback) è diverso dal thread di background che ha effettivamente servito o elaborato il task asincrono (ad esempio, il worker thread che ha letto materialmente il file dal disco o scaricato i dati dalla rete).

- Modello standard moderno: Questo schema rappresenta l'esatto modello di esecuzione delle moderne web application, sia sul lato client (es. i browser) che sul lato server (es. Node.js).

CALLBACKS AS CONTINUATIONS EXECUTED BY THE EVENT LOOP

- In this case:
 - events are implicit and refer to the completion with success or error of an asynchronous request/computations
 - event handlers are represented by continuation functions specified when invoking the asynchronous request/computation
 - *closures* are typically used to define the context to be used when processing the event
- Concrete examples - web app development / frameworks
 - client side: Javascript, DART
 - server side: node.js, vert.x, any reactor-based framework

JAVASCRIPT - CLIENT EXAMPLE

- Event-loop ruling script execution
 - in web apps, events concern the web page life-cycle, the DOM, the user input, network interactions, ... any event as defined by the HTML5 standard

- Example
 - Non blocking I/O

Può essere messa in qualsiasi punto del Handler perchè tanto event-loop deve eseguire tutto il codice prima di eseguire un altro Handler (quindi, anche se arriva prima la risposta della registrazione di cosa fare dopo la risposta, non è un problema)

```
...
let ajaxRequest = new XMLHttpRequest;
ajaxRequest.open('GET',url); HTTP GET
ajaxRequest.send(null);
ajaxRequest.onreadystatechange =
function() {
    // what to do next ...
}
```

Mandata HTTP Request (Questa è la chiamata asincrona primitiva. L'Event Loop la esegue e, facendolo, lancia un segnale al browser dicendo: "Avvia il download di questo URL". Fatto questo, l'Event Loop esce immediatamente e passa oltre.)

onreadystatechange è una proprietà dell'oggetto XMLHttpRequest a cui tu assegni una funzione (una callback). Il suo compito è definire "cosa fare dopo" quando l'operazione asincrona subisce un cambiamento di stato.

- wrong way of doing it...

```
// this is wrong
let ajaxRequest = new XMLHttpRequest;
ajaxRequest.open('GET',url);
ajaxRequest.send(null);
while (ajaxRequest.readyState === XMLHttpRequest.UNSENT){}
// !DEADLOCK!
```

perchè event-loop non vedrà mai il risultato. Lo vedrà al termine dell'esecuzione dell'handler perchè il risultato ha generato un evento nella coda che event-loop userà dopo per eseguire l'handler associato:

- L'evento di risposta è nella Coda.
- L'Event Loop non legge la Coda finché l'Handler non finisce.
- L'Handler non finisce perchè aspetta l'evento. = Punto di stallo (Deadlock dell'Event Loop).

JAVASCRIPT - CLIENT EXAMPLE

- setTimeout(callback, timeout) is another asynchronous function
 - registering a callback to be executed when the timeout expires
- What is the output of the following script?

```
for (let i = 1; i <= 3; i++){  
  console.log("setting timer..." + i);  
  setTimeout(() => { console.log(i); }, 0)  
}
```

Output:
setting timer...1
setting timer...2
setting timer...3
1
2
3

EXAMPLE IN NODE.JS

```
var sys = require("sys"),
    http = require("http"),
    url = require("url"),
    path = require("path"),
    fs = require("fs");
```

→ Registra un Handler che l'Event Loop eseguirà ogni volta che arriva una richiesta HTTP. Node.js prende questa callback e la associa a un Evento di Rete specifico (chiamato request).

```
http.createServer((request, response) => { Primo livello di callback
```

```
    let uri = url.parse(request.url).pathname;
```

```
    let filename = path.join(process.cwd(), uri);
```

```
    path.exists(filename, (exists) => { Secondo livello di callback: dopo che il file system (in background) ha finito di verificare se il file richiesto esiste sul disco, l'event-loop esegue callback
```

```
        if(exists) {
```

```
            fs.readFile(filename, (err, data) => { Terzo livello di callback: dopo che il Worker di background ha finito di leggere materialmente tutti i byte del file dal disco e li ha caricati in memoria, l'event-loop esegue callback
```

```
                response.writeHead(200);
```

```
                response.end(data);
```

```
            });
```

```
        } else {
```

```
            response.writeHead(404);
```

```
            response.end();
```

```
        }
```

```
    });
```

```
}).listen(8080); il Sistema Operativo tiene d'occhio la porta 8080.  
L'Event Loop di Node.js, nel frattempo, continua a girare. Se non arrivano richieste, l'Event Loop "dorme" o gestisce altri compiti.
```

```
sys.log("Server running at http://localhost:8080/");
```

CONTINUATION-BASED ASYNCHRONOUS PROGRAMMING: KEY POINTS

- Using a single thread to coordinate and manage multiple asynchronous task
 - eventually executed by external/OS-backed threads
 - occupazione di memoria ridotta low memory footprint compared to multi-threaded solutions (where a stack is used for each thread)
 - based on an implicit event-loop
- No low-level races
 - no shared state, only one thread accessing the state
- No low-level deadlocks
 - async handlers never block

ASYNC CONTROL FLOW PATTERNS

Se voglio fare prima un task poi un altro (callback innestate)

- **Sequence** task1 ; task2

```
asyncTask1(..., (res1) -> {  
    asyncTask2(..., (res2) -> {  
        ...}}}
```

Se voglio triggerare due task in parallelo (callback uno dopo l'altro)

- **Parallel** task1 | task2

Quando faccio nello stesso Handler più chiamate asincrone, l'ordine degli eventi con cui ricevo determina l'ordine di esecuzione delle callback. Non è deterministico l'ordine di esecuzione dei callback (non è detto che sempre si esegue il callback di asyncTask1/2 rispetto all'altro)

```
...  
asyncTask1(..., (res1) -> {  
    ...  
}  
asyncTask2(..., (res2) -> {  
    ...  
}
```

ASYNC PROGRAMMING: PROBLEMS

- Besides the benefits, there are some main well-known problems affecting CPS and event-driven programming, often referred altogether as *callback hell*

1 - asynchronous spaghetti

il funzionamento complessivo è

- ~~overall computation~~ fragmented into asynchronous handlers
- ~~this tampers~~ the modularity and structure of programs
 - making them difficult to understand and maintain

Questo compromette

2 - pyramid of doom / callback hell

- composition based on CPS leads to nested callbacks/continuations
- this increases complexity and leads to poor source code readability, bad reusability, bad extensibility

La callback hell avviene quando devi eseguire operazioni asincrone in serie (una dopo l'altra). Ogni operazione dipende dal risultato della precedente, costringendoti ad annidare le callback. Perché questo distrugge il codice?

- Leggibilità: Il codice si sposta sempre più a destra invece che verso il basso. Diventa difficile capire in quale scope ci si trovi.

- Gestione degli Errori: In questo schema, dovresti gestire l'errore per ogni singolo livello. Se dimentichi un `if(error)` in una delle callback intermedie, l'intero programma potrebbe fallire silenziosamente senza che tu sappia perché.

- Riutilizzabilità: Estrarre un pezzo di questa piramide per usarlo altrove è quasi impossibile perché è strettamente legato alle variabili catturate dalle closure esterne.

Nel modello sincrono il flusso è lineare e segue l'ordine di lettura dall'alto verso il basso. Nel modello CPS, la logica viene frammentata in più handler (le callback), introducendo due problemi critici: La logica di un singolo compito (es. "registrazione utente") non è più racchiusa in un unico blocco coeso, ma si disperde tra la funzione principale e le varie callback annidate. Inoltre, modificare l'ordine delle operazioni richiede di "ricablare" manualmente i collegamenti e i passaggi di parametri tra le funzioni, portando al fenomeno del callback hell.

2 PYRAMID OF DOOM | CALLBACK HELL

- Callback nesting

gli step sono asincroni, quindi ognuno ha callback (si rischia che il codice cresca in orizzontale anziché verticale)

```
step1(function(result1){
  step2(function(result2){
    step3(function(result3){
      // and so on ...
    })
  })
})
```

Appiattare queste piramidi, cioè renderle meno orizzontali, attraverso l'uso delle promise

“I love async, but I can’t code like this”

(a complaint of a developer on the Node.js Google Group)

PYRAMID OF DOOM

- In previous examples...

```
function loadUserPic(userId, ret) {  
  findUserById(userId, (user) => {  
    loadPic(user.picId, ret);  
  });  
}  
  
loadUserPic('john', (pic) => {  
  ui.show(pic);  
});
```

```
// Node.js  
  
var sys = require("sys"),  
    http = require("http"),  
    url = require("url"),  
    path = require("path"),  
    fs = require("fs");  
  
http.createServer((request, response) => {  
  var uri = url.parse(request.url).pathname;  
  var filename = path.join(process.cwd(), uri);  
  path.exists(filename, (exists) => {  
    if(exists) {  
      fs.readFile(filename, (err, data) => {  
        response.writeHead(200);  
        response.end(data);  
      });  
    } else {  
      response.writeHead(404);  
      response.end();  
    }  
  });  
}).listen(8080);  
  
sys.log("Server running at http://localhost:8080/");
```

PROMISES

Idea: ad ogni chiamata asincrona, invece di passare la continuazione (callback) alla funzione asincrona, la continuazione la voglio configurare nell'oggetto che mi restituisce subito la chiamata asincrona stessa, che sarebbe la promessa del risultato (simile alle future, concettualmente, ma qua non posso mai bloccarmi, come può capitare nelle future con la get)

- The callback hell can be partially solved by adopting the **promise** mechanism
 - originally proposed in 1976 by Daniel Friedman and D. Wise, as a proxy object that represents an unknown result ~~that is yet to be computed~~ ^{che deve essere ancora calcolato} (similar to futures)
- Promises represent ~~the eventual completion and result of a single asynchronous operation~~ ^{il possibile completamento e risultato}
 - they encapsulate asynchronous actions, ~~acting much like a value returned from the result of a computation~~ ^{funzionando in modo molto simile a un valore restituito con la differenza che} ~~only that the value may not be available at the time~~ ^{in quel momento}
 - a promise can be rejected or resolved once and only once
 - immutable once-resolved promises
- As a key property, **they allow for flattening callback nesting**

Una volta che ho la promise posso configurarla specificando qual'è il comportamento che voglio come callback quando è pronto il risultato, cioè quando la promise è stata risolta posso definire cosa fare (tramite then) in caso di successo/fallimento

PROMISES IN JAVASCRIPT

- Defined in Javascript standard (*)
 - also called *thenables* quando la promise è risolta (la promise ha uno stato interno), posso configurare la promise per definire cosa fare in caso di successo o fallimento
 - a developer uses *then* to attach callbacks to a promise when it is either fulfilled or an exception is realised.
- Supported by different frameworks (with some different semantics...)
 - Q - Most complete & mature,
 - When.js - Lightweight
 - jQuery Promises (not fully conform)

• Example

```
let promisedPic = loadUserPic('john');

promisedPic.then((pic) => {
  ui.show(pic);
});
```

(*) now in **ECMA-262 12th Edition - ECMAScript Language Spec (June 2021)**

<https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

GENERAL CASE

→ Tutte le chiamate asincrone ritornano una Promise

- An operation that may take some time return a Promise object at once and then complete its task in background (async):

```
let myPromise = asyncFunc(args)
```

- Promise is regarded as being a Promise to deliver a future value => we can configure the Promise to specify what should happen when the asyncFunc is complete

```
myPromise.then(onComplete, onError)
```

- onComplete => callback called if/when the asyncFunc is complete
- onError (optional) => callback called if/when the asyncFunc produced an error

THE PROMISE OBJECT

- A Promise object can be in one of three states:
 - **pending** L'operazione asincrona è stata delegata (al Pool di Worker) ma non è ancora finita.
 - when the Promise is first created
 - **resolved** (or *fulfilled*) Il compito è terminato con successo. Il valore promesso è ora disponibile.
 - if the async function / task ends correctly
 - **rejected** Qualcosa è andato storto
 - if the async function / task ends with an error
 - it is also called **settled** if it isn't pending
 - i.e. it is ^{or} either resolved or rejected
- Remarks
 - once a Promise is settled, its state and value does not change

I vantaggi nell'uso delle Promise al posto dei callback:

- Modularità: Ogni `.then` è un modulo separato e pulito.
- Gestione errori centralizzata: Il `.catch` cattura gli errori di qualsiasi livello precedente. Nel codice che abbiamo visto prima, avresti dovuto gestire l'errore in ogni singola callback.
- Compattezza: Il codice non "scivola" verso destra, ma resta leggibile dall'alto verso il basso.

THE PROMISE API IN JAVASCRIPT

- Creating a promise

```
let promise = new Promise((resolve, reject) => {  
  // do a thing, possibly async, then...  
  
  if (/* everything turned out fine */) {  
    resolve("Stuff worked!");  
  } else {  
    reject(Error("It broke"));  
  }  
});
```

- Attaching the callback

```
promise.then((result) => {  
  console.log(result); // "Stuff worked!"  
},(err) => {  
  console.log(err); // Error: "It broke"  
});
```

EXAMPLE

- Definition of an async function returning a Promise

il parametro passato alla Promise è una funzione con due parametri che sono a sua volta due funzioni (resolve e reject) che mi dicono chi devo chiamare quando ha successo la Promise o fallimento. Dopo, andando a chiamare la funzione delayWithRandom che restituisce una Promise, vado a configurare il caso di successo e fallimento passando questi due parametri al then della Promise. Anche se la funzione viene eseguita completamente prima di chiamare il then, comunque l'event loop deve aspettare il ciclo dopo per eseguire la continuazione della Promise (prende evento dalla coda dopo aver finito di eseguire il codice dell'Handler attuale). Ciò che viene messo nel ".then" è la logica del resolve e reject

```
function delayWithRandom(t,p){
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(() => {
        let r = Math.random();
        if (r > p) {
          resolve(r);
        } else {
          reject(r);
        }
      },t);
    });
  return promise;
}
```

- Calling the async function

Viene tutto eseguito da event-loop tranne il timeout che viene eseguito dal BG Worker che alla fine del timeout rimanda l'evento nella coda con il callback del timeout (se $r > p$, eseguo il then del successo, se no quello dell'errore)

```
let promise = delayWithRandom(1000,0.5);
promise.then((r) => {
  console.log("success: ",r);
},(r) => {
  console.log("fail: ",r);
});
```


PROMISE CHAINING

- Key feature for flattening the pyramid of doom
 - *then* method returns itself a promise, so that promises can be chained
- non è più un innestamento di callback, ma una concatenazione di promise in cui configuro mano a mano come voglio che le promise registrano (Registrare significa che stai popolando una "lista d'attesa" interna all'oggetto Promise. Quando la Promise cambia stato (da Pending a Settled), lei stessa si occupa di inviare le funzioni registrate all'Event Loop.)

```
let promise = new Promise(
  (resolve, reject) => {
    resolve(1);
  });

let p1 = promise.then((val) => {
  console.log(val); // 1
  return val + 2;
});

p1.then((val) => {
  console.log(val); // 3
});
```

```
let promise = new Promise(
  (resolve, reject) => {
    resolve(1);
  });

promise.then((val) => {
  console.log(val); // 1
  return val + 2;
}).then((val) => {
  console.log(val); // 3
});
```

Ciò che c'è dentro il then della Promise non viene eseguita da event loop, ma solo configurata (così da sapere cosa fare dopo che evento viene completato). Dopo che la Promise viene risolta (passa da Pending a Solved/Rejected) da Background Thread, essendo asincrona, il BG Thread genera evento che verrà preso da event loop ed eseguirà il then come continuazione della Promise.

PROPAGATING VALUES

The value of each Promise in a chain is whatever the previous `onComplete/onError` returns



```
let myProm1 = delayWithRandom(1000,0.5);
let myProm2 = myProm1.then((v) => {
  console.log(v);
  return "done";
});
myProm2.then((v) => {
  console.log(v);
});
```



Quello che ritorni (return) dentro un `.then()` diventa il valore risolto della Promise successiva.

Quando `myProm1` finisce con successo, viene eseguita la callback:

- Stampa il valore random `v`.
- Esegue `return "done"`.
- A questo punto, `myProm2` viene risolta istantaneamente con il valore "done".
- La callback collegata a `myProm2` riceve quindi "done" come argomento `v` e lo stampa.

CHAINING ASYNC FUNCTIONS

- To chain async functions (not only the first generating the Promise), *we can return Promises* in the chain

```
function delayWithRand(t){  
  let promise = new Promise(  
    (resolve,reject) => {  
      setTimeout(() => {  
        var r = Math.random();  
        resolve(r);  
      },t);  
    });  
  return promise;  
}
```

Nel chaining di then, posso
ritornare sia valori
primitivi/oggetti sia Promise
(il valore di essi, con return,
viene propagato al
prossimo then)

```
let promise = delayWithRand(1000);  
promise.then((r) => {  
  console.log("success: ",r);  
  return delayWithRand(1500);  
}).then((r) => {  
  console.log("success: ",r);  
  return delayWithRand(500);  
}).then((v) => {  
  console.log(v);  
})
```

SEQUENTIAL EXECUTION FLOW PATTERN

Grazie alla capacità delle Promise di essere concatenate, il flusso di esecuzione asincrono viene linearizzato, rendendo il codice molto più simile a quello sincrono tradizionale.

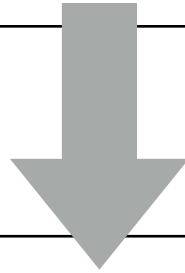
- Sequential execution flow pattern made easier

```
let myProm1 = asyncFunc1(...)
  .then((v) => {
    console.log(v);
    return asyncFunc2(...);
  }).then((v) => {
    console.log(v);
    return asyncFunc3(...);
  }).then(...);
```

FLATTENING THE PYRAMID

- Flattening the pyramid

```
findUserId('john').then((user) => {  
  return findPic(user.picId).then((pic) => {  
    ui.show(pic);  
  });  
});
```



```
findUserId('john')  
  .then((user) => {  
    return findPic(user.picId);  
  })  
  .then((pic) => {  
    ui.show(pic);  
  });
```

PARALLEL EXECUTION FLOW PATTERN

- Parallel execution flow pattern

```
let myPromise = asyncFunc1(...);  
  
myPromise.then((v) => {  
  console.log(v);  
  return asyncFunc2(...);  
});  
  
myPromise.then((v) => {  
  console.log(v);  
  return asyncFunc3(...);  
});
```

- Both the callbacks are called when `asyncFunc1` ends, with the same value hold by the Promise

PROMISIFYING EXISTING CODE

- (Example taken from <http://www.html5rocks.com/en/tutorials/es6/promises/>)

```
function get(url) {  
  // Return a new promise.  
  return new Promise((resolve, reject) => {  
    // Do the usual XHR stuff  
    let req = new XMLHttpRequest();  
    req.open('GET', url);  
  
    req.onload = () => {  
      // This is called even on 404 etc so check the status  
      if (req.status == 200) {  
        // Resolve the promise with the response text  
        resolve(req.response);  
      } else {  
        // Otherwise reject with the status text which will hopefully be a meaningful error  
        reject(Error(req.statusText));  
      }  
    };  
    // Handle network errors  
    req.onerror = () => {  
      reject(Error("Network Error"));  
    };  
    // Make the request  
    req.send();  
  });  
}
```

```
// configuring the promise..  
get('story.json').then((response) => {  
  console.log("Success!", response);  
}, function(error) {  
  console.error("Failed!", error);  
});
```

COMBINING PROMISES - ALL

→ Posso costruire una Promise da altre Promise

- Composing a set of independent asynchronous calls
 - to be executed in parallel, with a join point
- JavaScript Promise API: **all** method

```
let totalProm = Promise.all(promiseList);  
totalProm.then(function(values){...})
```

→ la funzione di callback viene chiamata quando tutte le Promises sono state risolte o almeno una è rejected

- *the callback is called when all Promises are resolved or at least ^{one} ~~on~~ is rejected*

- Example

```
let myProm1 = delayWithRand(1000);  
let myProm2 = delayWithRand(2000);  
  
let myTotalProm = Promise.all([myProm1, myProm2]);  
myTotalProm.then((values) => {  
  console.log(values[0]);  
  console.log(values[1]);  
});
```


COMBINING PROMISES - RACE

- Composing a set of independent asynchronous calls, but completing as soon as one completes la prima Promise che va in porto, la uso e delle altre non mi interessa
- JavaScript Promise API: **race** method

```
let totalProm = Promise.race(promiseList);  
totalProm.then(function(value){...},  
               function(value){...});
```

- *the callback is called when a Promise is resolved or rejected*
- Example

```
let myProm1 = delayWithRand(1000);  
let myProm2 = delayWithRand(2000);  
  
let myTotalProm = Promise.race([myProm1, myProm2]);  
myTotalProm.then((value) => {  
  console.log(value);  
});
```

PROMISE ERROR HANDLING

- Significant improvement ^{scrivendo} wrt callbacks

- **Rules** Queste regole descrivono il meccanismo di propagazione degli errori e dei successi nelle catene di Promise

- any handler, that returns a value and does not throw an error, passes on success to the next Promise, including onFailure handlers Se un handler (sia di successo che di errore) termina normalmente restituendo un valore, la Promise successiva nella catena passerà nello stato Risolta (Successo). Questo è il concetto di recupero dell'errore. Se intercetti un fallimento ma rimedi restituendo qualcosa, la catena torna a essere felice.
- any handler, that throws an exception, passes a reject on the next Promise Se dentro un qualsiasi handler (di successo o di errore) lanci un'eccezione con throw, la Promise successiva diventerà immediatamente Rifiutata (Errore).
- an unhandled state is passed to the next Promise in the chain
 - if there is no rejection handler and the Promise is rejected, then the state is passed to the next handler in the chain
 - once the state has found a handler then processing resumes its normal handler in the chain
 - similar to the synchronous case

Se si verifica un errore (o un successo) e l'anello corrente della catena non ha un handler per gestirlo, lo stato "salta" in basso fino a quando non trova qualcuno pronto a riceverlo. Le Promise non perdono per strada gli errori. Se non metti un handler di errore in un .then(), l'errore scorre verso il basso finché non incontra un handler di errore o un .catch().

Una volta che un errore ha trovato un handler che lo gestisce (senza lanciare nuovi errori), la catena si considera "guarita" e riprende l'esecuzione normale dei successivi handler di successo. È l'esatto equivalente del codice sincrono: una volta usciti da un blocco catch {}, il programma continua a eseguire le righe successive normalmente.

IMMEDIATELY RESOLVED PROMISE

- `Promise.resolve` API to build promises that can be immediately resolved
 - `then` can be used to chain a callback to be executed in next cycle of the event loop

```
console.log("this cycle");  
Promise  
  .resolve("done")  
  .then(() => {  
    console.log("next cycle");  
  });
```

`Promise.resolve("done")`: `Promise.resolve` crea una `Promise` che si trova già nello stato `Fulfilled` (completata con successo). Non c'è attesa di I/O o di calcoli in background.

TASKS AND MICROTASKS

I Microtask sono nati per gestire operazioni che devono essere eseguite "immediatamente dopo" il codice corrente, ma prima che il browser faccia altro (come aggiornare la grafica o gestire un nuovo Task).
- Esempi: Callback di `.then()`, `.catch()`, `.finally()` delle Promise e l'API `queueMicrotask()`.
- Comportamento: La Microtask Queue ha la priorità assoluta. Una volta che un Task è terminato, l'Event Loop svuota completamente la coda dei Microtask prima di passare al Task successivo.

- In JavaScript and Web world, the event queue - also called dispatch queue - hosts both tasks and *microtasks*

- event loop works as usual

- takes tasks one at a time and allows the task to run to completion before the next task is serviced

- tasks can also be preferentially dequeued (=> priorities)

- micro tasks have been introduced as a lightweight version of the task

- key feature: they are guaranteed to be executed before the next task starts
 - the microtask queue is processed after each task starts i.e. as soon as possible

- API do add micro tasks:

```
Promise.resolve().then(() => { ... });
```

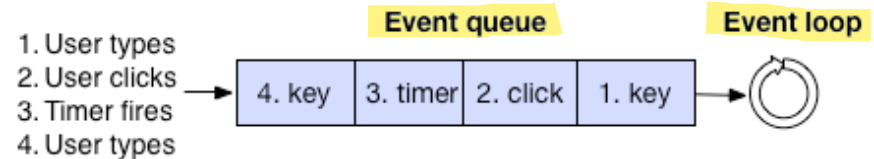
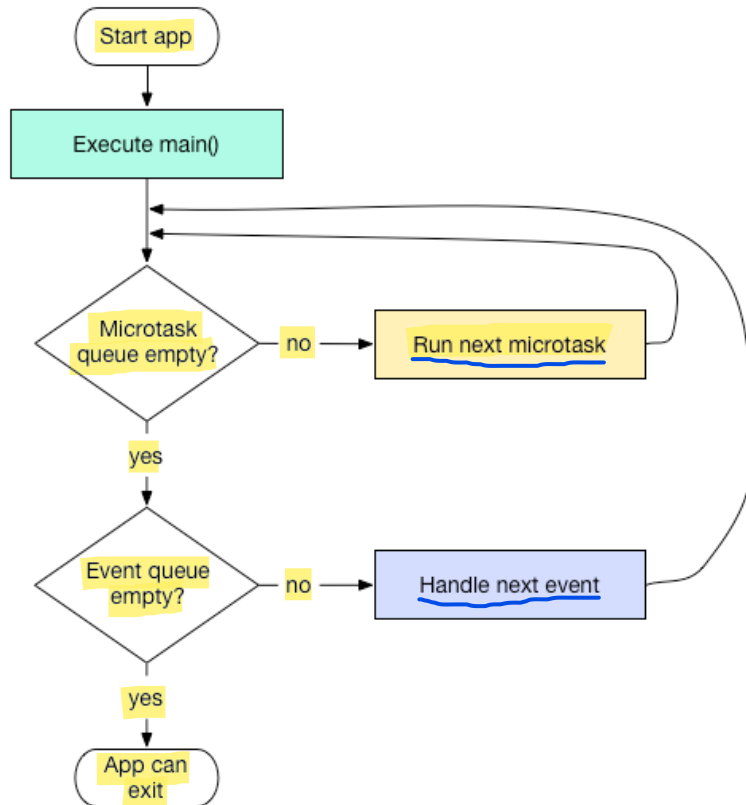
la distinzione tra Task e Microtask è ciò che permette al browser (o a Node.js) di dare priorità a certe operazioni rispetto ad altre, garantendo fluidità alla UI e velocità alle Promise.

Significa che l'Event Loop non è obbligato a svuotare la coda delle Task seguendo rigorosamente l'ordine di arrivo (il classico FIFO). Al contrario, può scegliere di estrarre e processare alcune Task prima di altre basandosi sulla loro priorità o sulla loro tipologia.

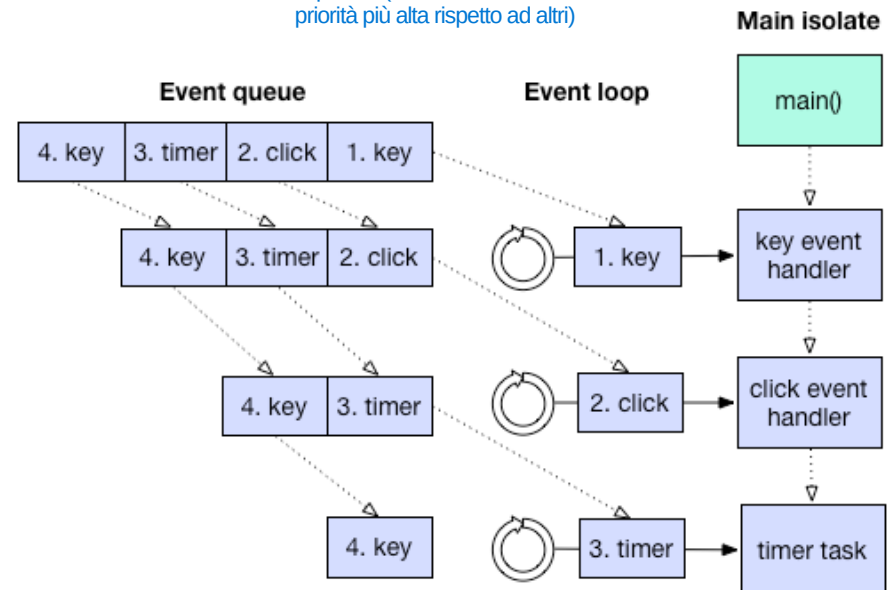
TASKS AND MICROTASKS - EVENT LOOP

(from <https://www.dartlang.org/articles/event-loop/>)

Qualsiasi Cloud App (codice nei Browser) segue questo modello ad eventi (asincrono ed event loop)



Event Queue organizzate con più livelli di priorità (a certi eventi viene dato una priorità più alta rispetto ad altri)



While the event loop is executing tasks from the microtask queue, the event queue is stuck: the app can't draw graphics, handle mouse clicks, react to I/O, and so on.

THE PROBLEM WITH PROMISES

1. ^{Foga (impazienza)} **Eagerness** and then parameter problem

- we cannot pass parameters to functions used in a then
- promises are ^{impazienti} **eager**: when they are constructed, they immediately begin work la valutazione delle Promise avviene non appena sono costruite.

Ecco perché quei .then non si comportano come vorresti se scritti male:

Esecuzione Immediata degli Argomenti
Quando scrivi una funzione, gli argomenti che passi tra le parentesi vengono valutati immediatamente, prima ancora che la funzione principale venga chiamata.

Guarda questo confronto:

Caso A (Funzione come riferimento): .then(printTime)
Qui stai passando solo il "nome" della funzione. L'Event Loop la tiene lì e la eseguirà solo quando la Promise precedente sarà risolta. Questo è sequenziale.

Caso B (Chiamata di funzione): .then(delay(1000))
Qui ci sono le parentesi (). JavaScript vede delay(1000) e lo esegue all'istante per ottenere un valore da passare al .then. Quindi il timer da 1000ms parte nel momento stesso in cui stai "configurando" la catena, non quando tocca a lui.

```
function delay(t){
  let promise = new Promise(
    function (resolve,reject){
      setTimeout(() => { resolve(); },t);
    });
  return promise;
}

function printTime(){
  var time = new Date().getTime();
  console.log(time);
}

printTime();
delay(1000)
.then(printTime)
.then(delay(1000))
.then(printTime);
```

Idea: aspetta un secondo, stampa il tempo, poi aspetta un'altro secondo e stampa il tempo

Parte subito perché ho messo le parentesi (). In JavaScript, quando chiamo la funzione, parte subito. Se vuoi che l'Event Loop lo faccia "dopo", devi impacchettarlo in una funzione senza eseguirla subito.

aspetto 1 secondo e poi genero evento che ha come callback il codice nel then successivo

```
...
printTime();
delay(1000)
.then(printTime)
.then(() => {
  return delay(1000);
})
.then(printTime);
```

Per rendere i .then davvero sequenziali, devi passare una funzione che crei la Promise solo quando viene invocata. In questo modo:

- L'Event Loop vede una funzione anonima () => { ... }.
- Viene configurata la resolve della Promise.
- Quando la Promise precedente è Resolved, l'Event Loop esegue la funzione anonima.
- Solo in quel secondo scatta delay(1000), creando la nuova Promise e il nuovo timer.

concettualmente, è una funzione asincrona delay che aspetta un certo tempo e poi esegue del codice

THE PROBLEM WITH PROMISES

2 • Cannot be used with loops/iterations

cioè creare then sequenziali in modo dinamico su una Promise

```
function delay(t){
  let promise = new Promise(
    (resolve, reject) => {
      setTimeout(function(){
        resolve();
      }, t);
    });
  return promise;
}
```

Il problema descritto è che le Promise sono "avide" (eager): appena le crei, partono. Se le metti in un ciclo for sincrono, le fai partire tutte insieme.

```
for (let i = 0; i < 3; i++){
  delay(1000).then(() => {
    let time = new Date().getTime();
    console.log(i + " > " + time);
  })
}
```

L'idea è semplice: crei una funzione che esegue un task, e nel suo .then() (la configurazione del successo) decide se richiamare se stessa per lo step successivo o fermarsi.

```
JavaScript
function sequenzaRicorsiva(i, max) {
  if (i >= max) return Promise.resolve(); // Caso base: abbiamo finito

  return delay(1000).then(() => {
    console.log(i + " > " + new Date().getTime());
    // RICORSIONE: restituiamo la chiamata a se stessa
    // Questo "registra" il prossimo step solo quando questo è finito
    return sequenzaRicorsiva(i + 1, max);
  });
}

sequenzaRicorsiva(0, 3);
```

it doesn't work as expected

Idea: Aspetta 1 secondo e stampa il tempo per 3 volte. Ma il risultato reale sarà molto diverso: non aspetto 3 secondi alla fine, ma solo 1 e dopo ricevo tre stampe perché vengono create 3 nuove Promise ed eseguite in parallelo.

Se io non so a priori gli step della concatenazione delle Promise (cioè quanti then concatenare), come faccio? Si usa la Ricorsione: a differenza del ciclo for sincrono (che "spara" tutte le chiamate insieme), la ricorsione asincrona rispetta l'Event Loop:

- Viene chiamata sequenzaRicorsiva(0).
- Viene creata la Promise del primo delay.
- L'Event Loop esce dalla funzione e aspetta.
- Solo quando il primo delay si risolve, viene eseguito il .then.
- Dentro il .then, viene chiamata sequenzaRicorsiva(1).
- Questa chiamata crea il secondo delay.

In questo modo, la "foga" (eagerness) delle Promise viene domata: la seconda Promise non nasce finché la prima non è morta.

Se non sai a priori quanti passi devi fare, non puoi scrivere una catena a mano. Devi usare la ricorsione asincrona.

Ecco come funziona la logica della funzione sequenzaRicorsiva(i, max) nella slide:

- Caso Base: Se $i \geq \text{max}$, restituisci una `Promise.resolve()`. La catena si ferma.
- Passo Ricorsivo: Chiami `delay(1000)`. Attacchi un `.then()`: il codice dentro il `.then()` verrà eseguito solo dopo 1 secondo. Dentro il `.then()`, chiami di nuovo `sequenzaRicorsiva(i + 1, max)`. Importante: Usi `return` davanti alla chiamata ricorsiva. Questo "aggancia" la Promise del passo successivo alla Promise del passo attuale.
- Perché funziona? A differenza del ciclo for (che è un'esplosione simultanea), la ricorsione asincrona crea una struttura a scatole cinesi: la seconda scatola viene aperta solo quando la prima ha finito di fare il suo lavoro.

THE PROBLEM WITH PROMISES

- 
- Cancellation
 - promises cannot be cancelled
 - no control over the async task
 - Possible waste of resources

FROM PROMISE TO ASYNC/AWAIT

- **async & await** language extension

Usando async and await per eliminare l'uso delle callback (Concettualmente, mando in esecuzione la chiamata asincrona, aspetta che la promise sia completata e con il risultato che ottieni dalla Promise esegui certe operazioni)

- most recent and promising solution in async programming
- goal: *providing a synch programming style while keeping an asynchronous core* (il motore 'sotto' deve essere sempre un event loop)

- Support

await non è una chiamata di metodo, ma è una keyword di linguaggio

- introduced in ES2017 (the 8th edition of the JavaScript Spec, released in June 2017)
- implemented in Microsoft .NET TAP (Task-based Asynchronous Pattern)
 - can be used in C# for instance
- Implemented in an increasing number of languages
 - Python, DART, ... language
 - Up-to-date list in <https://en.wikipedia.org/wiki/Async/await>

ASYNC WITH SYNCH STYLE

- Async style promoted by Promises

```
let p = asyncFunc(...);  
p.then(onCompleteFunc, onErrorFunc)
```

Eseguo il codice della Promise e poi quando ha terminato, nel prossimo ciclo del event-loop o nello stesso ciclo se rientra nei microtask, eseguo ciò che c'è nel then della Promise

- Main **async-await idea**: *reproducing a synchronous programming style, while keeping the asynchronous semantics*

```
let p = asyncFunc(...);  
let res = await p;  
<use res>
```

Sospendo l'esecuzione dell'handler nel punto in cui ho await e poi appena ho l'evento nella coda che mi dice che ha terminato l'esecuzione della funzione asincrona, riprendo dall'istruzione dopo await.

Quando il motore JavaScript incontra await p, accade qualcosa di molto diverso da una funzione normale:

- Non è un blocco del thread: Il thread principale non si ferma a "guardare l'orologio".
- È una sospensione: La funzione async in cui ti trovi viene "congelata". Il motore salva lo stato (variabili, punto di esecuzione) e restituisce il controllo all'Event Loop.
- L'Event Loop è libero: Mentre la tua funzione è sospesa, l'Event Loop può tranquillamente gestire altri eventi.

i.e., **conceptually wait for asyncFunc result** (~~cedendo~~ *yielding the control*) ~~and~~ *then continue with the program* (~~ottenendo~~ *getting back the control*) **when the promise is solved**

AWAIT

- Prima di await: Codice eseguito in modo sincrono fino alla sospensione.
- All' await: Il controllo torna all'Event Loop (fuori dalla funzione).
- Dopo l' await (Il tuo "GOTO"): Codice eseguito solo quando la Promise è risolta. Si trova nella riga successiva all'interno della funzione async (l'esecuzione della Promise viene passata ad un Background Thread che quando ha finito, genera evento nella coda che ha come Handler la continuazione dall'istruzione successiva al await, che continua esecuzione esattamente fino alla fine della funzione async dove si trova await, oppure finché non incontri un altro await.).

Il "salto" avviene verso la riga di codice che segue l' await, permettendo al programma di continuare la sua logica come se non ci fosse stata un'attesa, ma senza aver mai bloccato il thread principale.

Se faccio await di una Promise già risolta, comunque viene eseguita dall'Event Loop nel ciclo successivo (perché per sapere che è risolta deve beccare l'evento collegato ad esso)

• await operator

→ `await <promise>`

– can be applied to any Promise and *it suspends the function execution until the Promise is fulfilled*

– **it does not block the control flow** perchè non si può bloccare l'Event Loop

• the control flow is ^{sospeso} ~~yield~~, saving the current computation state

```
...  
console.log("here1");  
await delay(3000);  
console.log("here2");  
await delay(2000);  
console.log("here3");  
...
```

Allora, non viene sospeso l'Event Loop, ma viene sospeso il blocco che l'Event Loop sta eseguendo. Ovvero, definisco il blocco di codice che voglio sospendere (aggiungendo al blocco di codice la keyword async) con await, dando la possibilità all'Event Loop di continuare la sua esecuzione appena dopo il blocco che è stato sospeso.

IMPORTANTE: await vuole sempre la Promise; await non è associata alla funzione asincrona, ma alla Promise che la funzione asincrona restituisce

con await, l'Event Loop va a sospendere il blocco di codice che è stato etichettato come async e salta all'istruzione successiva al blocco async

AWAIT - ONLY INSIDE ASYNC

Una funzione `async` ha questa keyword non perchè restituisce una Promise, ma perchè deve delimitare il blocco di codice che deve essere sospeso dal momento in cui dentro c'è un `await` (poi ovviamente entrambi possono coincidere)

- `await` can be used only in the body of functions labelled as **async**

```
async function f(){  
  ...  
}  
  
async function f1() {  
  ...  
  await f();  
  ...  
}
```

```
var myFunc = async function() {  
  console.log("here1");  
  await delay(3000);  
  console.log("here2");  
  await delay(2000);  
  console.log("here3");  
  return 1+2;  
}
```

ASYNC FUNCTIONS

Dato che non ci sono più callback, ora questo costruito `await/async` posso usarlo nei cicli `for` (stile di programmazione sincrono con motore asincrono)

- **async functions are suspendible functions returning a promise**

```
var myFunc = async () => {  
  return 1 + 2;  
};
```

`async` restituisce, anche se non contiene l'`await`, una `Promise` per etichettare il fatto che comunque è asincrona (comunque, viene etichettata `async` perchè dentro usa `await`)

- The function is converted into an `AsyncFunction` object function
 - the function is just defined, not executed
- By calling the function, the code is executed as a normal function (in the same cycle) returning *a promise*

```
console.log("pre");  
console.log(myFunc);  
console.log(myFunc());  
console.log("post");
```

OUTPUT:

```
pre  
[AsyncFunction]  
Promise {}  
post
```

- `AsyncFunction` object contains the low-level machinery (based on *co-routines*, discussed later) to allow for expressing the body of the asynchronous function using a synchronous style

ASYNC FUNCTION WITH/WITHOUT AWAIT

- Note that **async function** can be called ^{sia} either with **await** ^{che senza} or not, producing a different behaviour
- Example

Ovunque definite await,
dovete etichettare con async

Con async/await non scompaiono le Promise perchè posso decidere se usare await o no, quindi posso ancora fare composizioni di Promise.

```
async function pause(t){
  console.log("before");
  let promise = new Promise(
    (resolve, reject) => {
      Bg worker setTimeout(() => {
        resolve(); setta la Promise in stato fulfilled
      }, t);
    });
  await promise;
  console.log("after");
}
```

Cosa succede sotto il cofano?

- "Sballamento" del valore: await estrae automaticamente il valore interno alla Promise. Se la Promise ha successo, ti restituisce direttamente il risultato; se fallisce, lancia un'eccezione che puoi catturare con un normale try / catch.
- Sospensione del contesto: L'esecuzione della funzione corrente viene letteralmente messa in pausa. Il codice scritto sotto la riga con await non viene eseguito finché la Promise non ha terminato il suo intero ciclo di vita.
- Nessun blocco del thread: Attenzione a non confonderti: l'Event Loop non si blocca! Solo la funzione in cui ti trovi viene sospesa. L'Event Loop è libero di uscire, fare altri giri, gestire click dell'utente o eseguire altre Macro-task mentre aspetta che il BG Worker finisca quel lavoro.

la continuazione

without

```
console.log("before call");
pause(1000); viene restituita la Promise
console.log("after call");
```

with

devo creare una funzione async
così che io posso usare await

```
async function main(){
  console.log("before call");
  await pause(1000); aspetto il completamento della Promise (che verrà completata da un BG Worker, etc..)
  console.log("after call");
}
main();
```

AWAIT: DETAILS

- `await <promise>` converts a Promise into a result or an exception *just as if it was a result from a synchronous function*
 - if the Promise is resolved, the await operator extracts the resolved value and returns it
 - if the Promise is rejected then the await throws an exception with the rejected value

AWAIT AND CO-ROUTINES

Normalmente, quando una funzione fa return, "muore" e sparisce dalla memoria. Con await, la funzione esegue un ritorno prematuro ma rimane viva. Cosa succede: Appena il thread incontra await, la funzione si interrompe. Il thread "esce" dalla funzione e torna da chi l'aveva chiamata. Il "pacchetto" ricevuto: Il chiamante non riceve il valore finale (che non è ancora pronto), ma riceve un oggetto Promise in stato pending. Questo oggetto è la ricevuta che promette: "Ti darò il risultato non appena avrò finito".

- ^{si comporta come} ~~await behave like~~ a return that can be resumed
 - ^{programma chiamante} ~~when a function executes an await,~~ the thread returns to the calling program which receives the Promise object created by the async function in a pending state
 - the state of the computation is saved, to be restored when the control is back
 - the async function is resumed asynchronously, as soon as the event-loop thread ^{recupera} ~~fetches~~ the corresponding event in the event queue
- To release and resume the control in the implementation of await/async, the **co-routine** mechanism is exploited
 - old mechanism adopted for cooperative scheduling
 - brought back in asynchronous programming based on a single-control flow
 - more at the end of this module

Se eseguo await di una funzione asincrona in un blocco async e ho del codice fuori dal blocco async, l'event loop esce dal blocco async dove si trova await ed esegue il codice fuori dal blocco async. Dopo che il BG worker ha completato il task asincrono, tramite evento, l'event loop riprende dove si era sospeso ed esegue il restante codice

Il termine "Co-routine" indica una funzione che può sospendere la propria esecuzione e riprenderla in un secondo momento dal punto esatto in cui si era fermata.

Scheduling Cooperativo: Si chiama "cooperativo" perché è la funzione stessa che decide di "sospendersi" usando la keyword await. Non è il sistema operativo a strapparle il controllo (come nel multithreading classico), ma è la funzione che dice: "Ho finito per ora, svegliatemi quando la Promise è pronta".

BENEFITS

- Asynchronous programming greatly simplified
 - no need to explicitly use Promises/then/chains for implementing the sequence execution pattern
 - eagerness & then parameter problem solved
 - can be used with loops/iterations...

```
async function waitFor(t){
  console.log("waitFor - pre");
  await delay(t);
  console.log("waitFor - post");
}

async function main(){
  console.log("Before");
  for (let i = 0; i < 3; i++){
    await waitFor(1000);
    console.log("Step "+i);
  }
  console.log("After");
};
```

ISSUES

- Some issues affecting async/await
 - await can be used only inside an async function body => it cannot be used at the top level
 - need to introduce at least one main async function
 - an async function cannot resume while the thread is busy running other handlers
 - the event-loop semantics is preserved
 - **blocks {...} are no more guaranteed to be atomic**
 - the execution of the code of a single block {...} can span over multiple iterations of the event loop
 - => **race conditions**
 - composing await/async => **Need to mix await/async with Promises API**

la parte prima della await viene eseguita nel primo ciclo; la parte che viene dopo in un secondo ciclo (è sempre l'Event Loop che esegue le varie parti)

logiche

Non ci sono corse critiche di basso livello (perché è sempre l'Event Loop che modifica il proprio stato), però io potrei vedere che quello stato viene modificato in più cicli nonostante veda un solo blocco computazionale (si spezza atomicità)

Ci sono due tipi di Race conditions:

- 1. Race Condition di Basso Livello (Data Race): Queste avvengono nel mondo del multithreading reale. Qui, più thread fisici accedono alla stessa cella di memoria nello stesso identico istante.

Esempio: L'istruzione x++ sembra una sola, ma a basso livello sono tre: (1) Leggi valore, (2) Incrementa, (3) Scrivi valore.

Conseguenza: Se due thread fanno x++ insieme, potrebbero leggere entrambi 10, incrementare a 11 e scrivere 11. Il risultato corretto sarebbe 12, ma un incremento è andato perso.

In JavaScript: Questo non può accadere per le variabili JS standard, perché l'Event Loop garantisce che solo un pezzetto di codice alla volta tocchi la memoria.

- 2. Race Condition di Alto Livello (Race Condition Logica): Queste sono quelle che affliggono JavaScript e le funzioni async/await. Non avvengono perché due istruzioni "si scontrano" elettricamente nella RAM, ma perché l'ordine logico delle operazioni viene rimescolato dall'Event Loop.

Il problema: L'interferenza avviene tra i "pezzi" di una funzione separati da await.

Il colpevole: È il programmatore che assume che nulla cambi tra una riga e l'altra, dimenticando che l'await ha ceduto il controllo al mondo esterno.

Esempio (Il bancomat): Controlli se ci sono soldi (OK). await (ti sospendi). Nel frattempo, un altro handler viene eseguito e svuota il conto. Ti risvegli e prelevi (Errore: il conto ora è vuoto, ma tu avevi già passato il controllo!).

COMPOSITION ISSUE: EXAMPLE

- Composing await/async => Need to mix await/async with Promises API

```
async function f(){
  await delay(1000);
  return 1+2;
}

async function g(){
  await delay(5000);
  return 2+3;
}

async function compose(){
  console.log("before");
  let r = await Promise.all([f(),g()]);
  console.log("done: "+r);
}

compose();
```

async/await non mi eliminano le Promise, vengono integrate (ho ancora bisogno di avere come elemento concettuale le Promise)

"DESIGN CLASH" ISSUE

Conflitto di Progettazione

Design clash between synchronous and asynchronous

- strong discipline needed in mixing synch/asynch style to avoid programs whose behaviour is difficult to understand
- *if we want to mix/integrate them, what are the key abstraction concepts / rationale that we can use to think /design a program, besides the programming mechanisms?*
- *What about modularity? What impact on reuse and extensibility?*

Need of higher level of abstractions/concepts ^{che integrino naturalmente} ~~that naturally integrate~~
^{una prospettiva sincrona e una asincrona} ~~a sync and an async perspective~~

Il conflitto nasce dal fatto che i due stili impongono vincoli opposti sul flusso del tempo e del controllo:

- Stile Sincrono: Il tempo coincide con lo spazio del codice. Se la riga B è sotto la riga A, B viene eseguita dopo che A ha finito. La funzione che chiama ha il controllo totale.
- Stile Asincrono: Il tempo è scorrelato dallo spazio. La riga B sotto la riga A viene eseguita mentre A sta ancora lavorando. Il controllo viene delegato all'esterno.

Se mischi i meccanismi puri (es. ficcare callback o Promise a caso dentro funzioni sincrone), rompi il contratto dell'interfaccia. Una funzione che sembra sincrona dall'esterno restituisce un valore "vuoto" perché l'elaborazione interna non è ancora terminata. Il comportamento del sistema diventa imprevedibile e impossibile da simulare mentalmente.

La slide solleva tre domande architettonali cruciali:

- La Modularità viene avvelenata (Contagio Asincrono): L'asincronia è contagiosa. Se un modulo interno (es. il modulo di accesso al database) diventa asincrono, tutti i moduli che lo chiamano devono diventare asincroni per poterne usare il risultato. Non puoi più isolare l'asincronia in un singolo "cassetto" (modulo) stagno.
- Crollo del Riuso (Reuse): Un componente puramente sincrono (es. un algoritmo di validazione dati) non può essere riutilizzato facilmente in un contesto asincrono senza essere riscritto o avvolto in strutture complesse.
- Barriere all'Estensibilità: Se vuoi estendere un sistema aggiungendo una nuova funzionalità (es. un plugin che salva i dati anche su cloud invece che solo in memoria), trasformi un'operazione istantanea in una che richiede tempo. Se l'architettura iniziale non ha previsto questa natura temporale, estendere il codice richiede di rifattorizzare l'intera catena di chiamate.

- Pensiero Sincrono: È come leggere una ricetta. Fai il punto A, poi il punto B, poi il punto C. Tutto è prevedibile e nulla cambia mentre stai leggendo.
- Realtà Asincrona: È come gestire un ristorante. Mentre aspetti che l'acqua bolla (await), vai a prendere le ordinazioni, ma nel frattempo un altro cliente potrebbe aver finito il sale che ti serviva.

Il clash è questo: Cerchiamo di scrivere codice che sembra una ricetta lineare (async/await), ma in realtà stiamo gestendo un ristorante caotico dove il "mondo" cambia sotto i nostri piedi mentre aspettiamo.

MODULARITY/ENCAPSULATION ISSUE

Siccome f() è asincrona, se vuoi che g() aspetti che f() finisca, sei obbligato a trasformare anche g() in async e aggiungere await. Cosa significa questo per il Design?

- Modularità distrutta: Se hai una libreria di 100 funzioni sincrone e una sola funzione in fondo alla catena diventa asincrona (magari perché devi leggere un dato da un database), devi cambiare tutte le 100 funzioni sopra di lei per farle diventare async.

- Incompatibilità: Non puoi usare una funzione asincrona dentro un'interfaccia che si aspetta un risultato immediato (sincrono). L'asincronia "inquina" tutto ciò che tocca.

```
function delay(t){
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(function(){
        resolve();
      },t);
    });
  return promise;
}
```

```
async function f(){
  console.log("started f()")
  await delay(2000);
  console.log("done f()");
}
```

```
function g(){
  console.log("started g");
  f();
  console.log("done g");
}
```

```
g();
```

this does not
work as expected



```
function delay(t){
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(function(){
        resolve();
      },t);
    });
  return promise;
}
```

```
async function f(){
  console.log("started f()")
  await delay(2000);
  console.log("done f()");
}
```

```
async function g(){
  console.log("started g");
  await f();
  console.log("done g");
}
```

```
g();
```

ABOUT COROUTINES

- Subroutine (Funzione classica): Ha un inizio e una fine. Quando la chiami, parte; quando arriva al return, muore e "dimentica" tutto. Segue il modello LIFO (Last-In-First-Out) dello stack.
- Coroutine: È una funzione "con memoria". Può decidere di sospendersi (yield o await), restituire il controllo e poi riprendere esattamente da dove era rimasta, mantenendo intatte le sue variabili locali.

- Generalisation of the subroutine concept ^{che consente di sospendere e riprendere l'esecuzione} by allowing ~~execution to be suspended and resumed~~ [Con63]

- for non-preemptive, cooperative multitasking
- basic building block for implementing cooperative tasks, exceptions, event loops, iterators, infinite lists and pipes
- not related to events and event-driven programming

Le coroutine sono nate per lo scheduling cooperativo: un modo per far girare più programmi su un solo processore senza che il sistema operativo debba "forzare" il cambio (multitasking non-preemptive). Erano i programmi stessi a "cedersi il passo" a vicenda.

Oggi le usiamo per gestire gli eventi (I/O, timer), ma la tecnologia della coroutine in sé è solo un modo per gestire il flusso di esecuzione, indipendentemente dal fatto che ci sia un evento o meno.

- Historically, coroutines originated as an assembly language method, but are supported in some high-level programming languages.
 - early examples include Simula, Smalltalk, and Modula-2
- The mechanism has received a renewed attention in recent times mainly because of async programming
 - more recent examples are Ruby, Lua, Julia, Go, Kotlin
 - Up-to-date view: <https://en.wikipedia.org/wiki/Coroutine>

COROUTINES AS GENERALISED ROUTINE

- When calling a coroutine, a *coroutine can exit by calling other coroutines*, which may later return to the point where they were invoked in the original coroutine
 - from the coroutine's point of view, it is not exiting *but calling another coroutine*
 - calling another coroutine can be done by *cedendo il controllo ad essa* "yielding" to it

```
var q := new queue

coroutine produce
  loop
    while q is not full
      create some new items
      add the items to q
      yield to consume

coroutine consume
  loop
    while q is not empty
      remove some items from q
      use the items
      yield to produce
```

A cooperative one-flow based producer consumer system (from <https://en.wikipedia.org/wiki/Coroutine>)

COROUTINES CHARACTERISTICS

- A coroutine instance holds state, and varies between invocations
 - there can be multiple instances of a given coroutine at once
- Traditional *subroutines* are special cases of coroutines
 - any subroutine can be translated to a coroutine which does not call yield
- Can be easily implemented in languages natively supporting continuations
 - it *cannot* be implemented just as a library on top of a language based on just subroutines and subroutine-like mechanism (e.g. methods)

Perché? Le funzioni standard usano lo Stack. Quando una funzione finisce, lo stack viene svuotato ("pop"). Non c'è modo per una funzione standard di dire allo stack: "Sospendimi, tieni in caldo le mie variabili e fammi tornare tra 5 minuti".

Una subroutine è una coroutine che non si mette mai in pausa. Entra, lavora e muore. Per questo motivo, ogni programma scritto con funzioni normali può essere pensato come un programma a coroutine "pigre". La coroutine è il concetto più potente e flessibile che include il precedente.

A differenza di una funzione normale che "riparte da zero" ogni volta, una coroutine è come un'istanza di un oggetto.

- Stato Persistente: Ogni volta che la funzione viene sospesa (con await), il motore salva il valore di tutte le variabili locali. Quando viene ripresa, non deve ricalcolare nulla: sa esattamente dove si trovava.

- Istanze Multiple: Puoi lanciare la stessa funzione async tre volte contemporaneamente. JavaScript creerà tre "istanze di coroutine" separate, ognuna con il suo stato e la sua velocità di esecuzione. Non si confondono tra loro.

CONCURRENCY MODEL BASED ON COROUTINES

Il punto centrale è il passaggio dai Thread "pesanti" (gestiti dal Sistema Operativo) ai Thread "leggeri" (gestiti dal linguaggio stesso).

- Coroutines can be used as low-level mechanism to implement *lightweight threads*
 - one thread can have many coroutines inside it, but only one coroutine is in execution at a given time
- Fast and cheap to create and to manage
 - not requiring context switching by the OS
- Kotlin language example - main concepts:
 - Suspending computations
 - Coroutine dispatchers
 - Coroutine builders



Una singola applicazione può far girare 100.000 coroutine contemporaneamente su un unico thread reale, consumando pochissima RAM (pochi byte per salvare lo stato delle variabili). In un thread possono esserci molte coroutine, ma solo una alla volta sta effettivamente usando la CPU. Le altre sono "parcheggiate" in attesa.

A TOY EXAMPLE

```
import kotlinx.coroutines.*

fun main(args: Array<String>) = runBlocking {
    println("before async call")
    val result = async {
        println("inside the async call")
        delay(1000)
        println("exiting the async call")
        100
    }
    println("after the async call, before greet")
    greetDelayed(200)
    println("after greet trigger")
    println("${result.await()} ")
}

suspend fun greetDelayed(delayMillis: Long) {
    delay(delayMillis)
    println("Hello, World!")
}
```

OUTPUT

```
before async call
after the async call, before greet
inside the async call
Hello, World!
after greet trigger
exiting the async call
100
```

SUSPENDING COMPUTATIONS IN KOTLIN

- Computations that can suspend their execution without blocking the thread in which they reside
 - a suspending computation allows the thread to be used for other computations until they need to be resumed
- Types
 - **suspending functions**
 - suspending computations in the shape of a function: modifier `suspend`
 - **suspending lambda**
 - like a normal lambda, but can suspend its own execution by calling other suspending functions

SUSPENDING FUNCTIONS: EXAMPLE

- Example

```
suspend fun greetAfter(name: String, delayMillis: Long) {  
    delay(delayMillis)  
    println("Hello, $name")  
}
```

- the execution of `greetAfter()` will be suspended when `delay()` is called
- `delay()` is a suspending function itself, suspending the execution for a give duration
- once `delay()` has been completed, `greetAfter()` will resume its execution normally
- and while `greetAfter()` is suspended, the thread of execution may be used to do other computations

COROUTINE DISPATCHERS & BUILDERS

- **Coroutine dispatchers**
 - in order to decide what thread to start or resume a coroutine on a coroutine dispatcher is used.
- **Coroutine builders**
 - a function that takes a suspending lambda and creates a coroutine to run it
 - coroutine builders provided by Kotlin:
 - `async()` - start a coroutine when a result is expected
 - `launch()` - start a coroutine that doesn't return a result
 - `runBlocking()` - bridge blocking code into suspendable code

RELATED TO COROUTINE: FIBERS

Dibattito aperto sul fatto di dire, è preferibile usare un approccio ad eventi oppure un approccio che mantiene l'idea di Thread

- A **fiber** is a lightweight thread of execution
 - like threads, **fibers share address space**
 - differently from threads, **fibers use *cooperative multitasking* while threads use preemptive multitasking**
 - if threads typically depend on the kernel's thread scheduler to preempt a busy thread and resume another thread, **fibers ~~yield themselves~~ to run another fiber while executing**
 - so can be implemented using a single thread
- Fibers are like coroutines, but at different level of abstraction
 - **coroutines are a language-level construct**, a form of control flow
 - **fibers are a systems-level construct**, viewed as threads that happen to not run in parallel
 - **fibers may be viewed as either an implementation of coroutines or as a substrate on which to implement coroutines**
- **Implemented by several OS/frameworks/languages**
 - e.g. UNIX/pthreads, Windows/.NET
 - **Java: *virtual threads*** (from JDK 19)

La differenza fondamentale tra Fibers (multitasking cooperativo) e Threads (multitasking preemptive) risiede in chi ha il potere di decidere quando un compito deve fermarsi per lasciar spazio a un altro (programma vs OS: interruzioni forzate da OS vs volontarie del programmatore).

cedono il controllo per consentire

su cui

È un'unità di esecuzione. Puoi pensare alla Fiber come al "veicolo" su cui viaggia la Coroutine. Le fibre si comportano come thread che però hanno deciso di non correre mai in parallelo sullo stesso core, ma di darsi il cambio ordinatamente.

In alcuni linguaggi, come Java, ha scelto di mantenere una visione in cui si riesce a risolvere i problemi associati alla reattività introducendo i Virtual Thread (non modificando il linguaggio). Usano Virtual Thread, invece di implementare async/await

VIRTUAL THREADS VS. ASYNC PROGRAMMING: A NEVER ENDING DEBATE

Nell'asincronia, se una funzione è async, tutte le funzioni che la chiamano devono diventare async. Con i Virtual Threads, una funzione è solo una funzione. Il codice è più pulito, più facile da testare e puoi usare tutte le vecchie librerie nate prima dell'asincronia. Il controllo del flusso (if, while, for) rimane naturale.

- See the recent overview by Brian Goetz about Virtual Threads [Goe22]

— virtual threads provides a better modularity and encapsulation for modularising logical processes and control flows (vs. async programming...)

— ~~however~~ ^{tuttavia} virtual threads — like physical threads — do not provide a first-class treatment for *events* and reactive computations

- must be implemented and managed at the application level..

I thread (anche quelli virtuali) sono nati per eseguire processi sequenziali.

- Async: È costruito sopra gli eventi. Se arrivano 3 messaggi diversi, l'Event Loop è bravissimo a smistarli in modo reattivo.
- Virtual Threads: Non hanno un concetto "nativo" di evento. Se vuoi che un thread virtuale reagisca a un evento, devi farlo "mettere in attesa" su una coda. La gestione degli eventi diventa qualcosa che devi implementare tu a livello applicativo (es. usando code o semafori), mentre in Node.js è il cuore del motore.

Significa poter isolare un processo logico (un compito) in un'unità autonoma che si comporta come se avesse il pieno controllo della CPU.

REFERENCES

- Trevor Burnham. AsyncJavascript. The Pragmatic Programmers Series
- Event-based Concurrency Control. Tom Van Cutsem, Vrije Universiteit Brussel. Tutorial at OOPSLA 2009. Slides available at http://soft.vub.ac.be/~tvcutsem/talks/presentations/T37_nobackground.pdf
- Programming Without a Call Stack – Event-driven Architectures. Gregor Hohpe. www.eaipatterns.com
- MSDN - Asynchronous programming on .NET 4.5
- Stefan Tilkov and Steve Vinoski. 2010. Node.js: Using JavaScript to Build High-Performance Network Programs. IEEE Internet Computing 14, 6 (November 2010), 80-83.
- Ian Elliot. JavaScript Async: Events, Callbacks, Promises and Async Await. I/O Press
- EcmaScript
 - <https://www.ecma-international.org/ecma-262/6.0/>
 - <https://www.ecma-international.org/ecma-262/8.0/>

REFERENCES

- **[Con63]** Conway, M. E. (July 1963). "Design of a Separable Transition-Diagram Compiler". *Communications of the ACM*. 6 (7): 396–408.
- **[Mou04]** Ana Lucia de Moura; Roberto Ierusalimschy (2004). "Revisiting Coroutines". *ACM Transactions on Programming Languages and Systems*. 31 (2): 1–31. CiteSeerX 10.1.1.58.4017. doi:10.1145/1462166.1462167.
- **[Pro18]** Aleksandar Prokopec. Theory and Practice of Coroutines with Snapshots. ECOOP 2018
- **[Goe22]** Brian Goetz. Virtual Threads: New Foundations for High-Scale Java Applications. InfoQ (Sept. 22)
 - <https://www.infoq.com/articles/java-virtual-threads/>
- "Holy crap: JVM has coroutine/continuation/fiber etc."
 - <https://web.archive.org/web/20150319052055/http://weblogs.java.net/blog/forax/archive/2009/11/19/holy-crap-jvm-has-coroutinecontinuationfiber-etc>